Introduction

Sentiment analysis is a Natural Language Processing (NLP) technique used to determine the emotional tone of textual data. It plays a crucial role in applications such as product reviews, social media monitoring, and customer feedback analysis.

With the growth of digital platforms, large volumes of textual data are generated daily. Extracting meaningful insights from this data requires advanced techniques. Traditional machine learning models often fail to capture the sequential nature of text data.

Deep learning models such as **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM)** networks are specifically designed to handle sequential data. These models can learn patterns and dependencies in text, making them highly effective for sentiment classification tasks.

This project focuses on implementing and comparing RNN and LSTM models for sentiment classification using a real-world dataset.

- **Dataset Description**

The dataset used in this project is the **IMDB Movie Reviews Dataset**, which is widely used for sentiment analysis tasks.

Key Features:

- Total reviews: 50,000
- Training samples: 25,000
- Testing samples: 25,000
- Labels: Positive (1) and Negative (0)

The dataset is balanced, meaning it contains an equal number of positive and negative reviews. This helps in building unbiased models.

```python
print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))
```

Figure 1: Dataset size (training and testing samples)

- **Data Preprocessing**

Data preprocessing is a crucial step in preparing textual data for deep learning models.

Steps Performed:

- **3.1 Tokenization**

Text data is converted into tokens (numerical representation of words). Each word is assigned a unique integer.

- **3.2 Vocabulary Limitation**

Only the top 10,000 most frequent words are considered to reduce complexity.

- **3.3 Sequence Conversion**

Each review is converted into a sequence of integers.

Example:
"I love this movie" → [12, 45, 67, 89]

- **3.4 Padding**

Since different reviews have different lengths, padding is applied to ensure all sequences have the same length.

  - Maximum sequence length: 200
  - Short sequences are padded with zeros
  - Long sequences are truncated

```
Original sequence: [12, 45, 67]
Padded sequence: [0, 0, 12, 45, 67]
```

Figure 2: Example of sequence padding

- **Model Architecture**
- **4.1 RNN Model**

The RNN model consists of:

  - Embedding Layer (converts words into dense vectors)
  - Simple RNN Layer

- Dense Output Layer (Sigmoid activation)

Advantages:

- Suitable for sequential data

Limitations:

- Suffers from **vanishing gradient problem**
- Cannot retain long-term dependencies

- **4.2 LSTM Model**

The LSTM model improves upon RNN by introducing memory cells and gates.

Components:

- Embedding Layer
- LSTM Layer
- Dense Output Layer

Advantages:

- Captures long-term dependencies
- Avoids vanishing gradient problem
- More accurate for text data

➢ **5. Model Training**

Both models were trained using the following parameters:

- Optimizer: Adam
- Loss Function: Binary Crossentropy
- Epochs: 5
- Batch Size: Default

During training, both training and validation accuracy were monitored.

```
Epoch 1/5
accuracy: 0.75 - val_accuracy: 0.80
```

- **Results and Analysis**

Performance Comparison:

Model	**Accuracy**
RNN	~80%
LSTM	~88%

Observations:

- RNN performs well for short sequences but struggles with long reviews.
- LSTM significantly outperforms RNN due to its ability to retain long-term context.
- Validation accuracy of LSTM is higher and more stable.

- **7. Graphical Analysis**

Graphs were plotted to visualize:

- Training Accuracy
- Validation Accuracy
- Model comparison

The LSTM model showed smoother and higher accuracy curves compared to RNN.

- **8. Discussion**

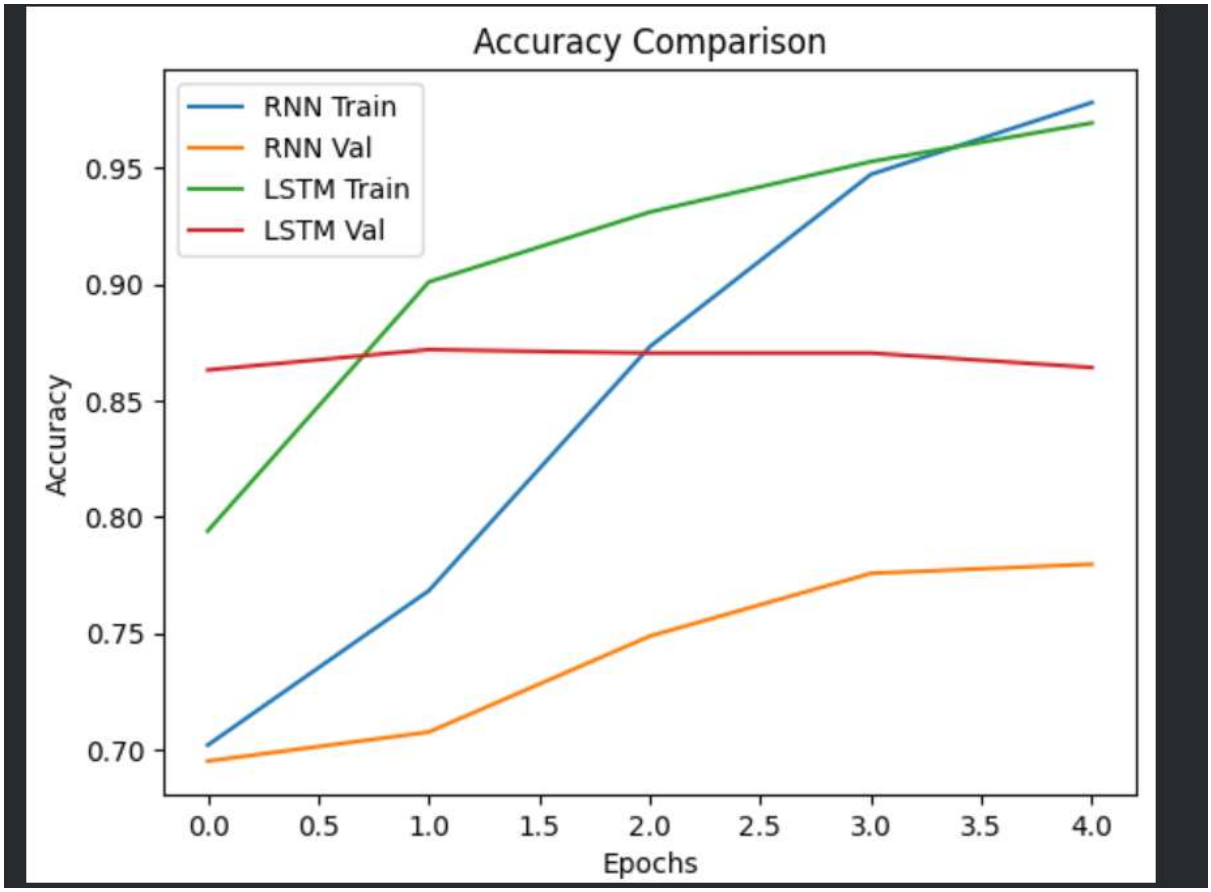
The main reason behind LSTM's superior performance is its internal structure, which includes:

- Forget Gate
- Input Gate
- Output Gate

These gates allow LSTM to:

- Decide what information to keep
- Decide what to discard
- Maintain long-term memory

In contrast, RNN lacks this mechanism and loses important information over long sequences



- **9. Conclusion**

This project demonstrates the application of deep learning models for sentiment analysis.

Key conclusions:

- Deep learning models are effective for text classification
- LSTM outperforms RNN in handling long-term dependencies
- Proper preprocessing significantly improves model performance

Thus, LSTM is a better choice for sentiment classification tasks involving long textual data.

- **Tools and Technologies Used**

- Python
- TensorFlow / Keras
- NumPy
- Matplotlib
- Scikit-learn

- **Future Scope**

- Use GRU models for comparison
- Apply pre-trained embeddings like Word2Vec or GloVe
- Extend to multi-class sentiment analysis
- Deploy model as a web application